

2- Notions de programme, processus, thread et ressource partagée.

1. Notion de programme

Un programme est un ensemble d'opérations destinées à être exécutées par un ordinateur. Il peut être sous plusieurs formes :

- a) Un programme source : Le code écrit dans un langage de programmation (C, Java, Python....), sous forme d'un fichier texte. Il peut être compilé vers une forme binaire ou directement interprété.
- b) Un programme binaire (executable) décrit les instructions à exécuter par un microprocesseur. Ces instructions définissent un langage machine.

2. Notion de processus : Un processus est un programme en cours d'exécution. Il est caractérisé par :

- Un numéro d'identification unique (PID);
- Un espace d'adressage (code, données, piles d'exécution);
- Un état principal (prêt, en cours d'exécution, bloqué, ...);
- L'état des registres (CO, PSW, Sommet de Pile...);
- Une priorité;
- Les ressources allouées (fichiers ouverts, mémoires, périphériques ...);
- Autres informations

2.1. Changement de contexte :

Après une interruption on sauvegarder le contexte du processus en cours d'exécution en mémoire (usuellement dans une zone mémoire appelée PCB). On recherche le contexte d'un autre processus en mémoire. pour le restaurer dans le processeur.

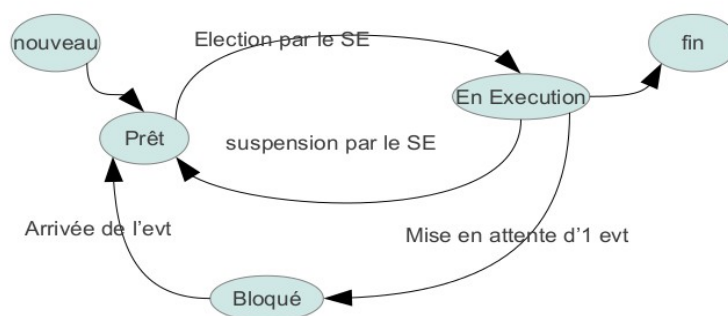


figure : Graphe d'états de processus

2.2. États d'un processus : Lors de la création d'un nouveau processus il passe à l'état prêt. À l'état **prêt** les processus attendent le processeur. Un seul processus est élu par le SE pour passer à l'état **En exécution** (appelé aussi actif ou élu) pour s'exécuter au niveau du processeur. Après une interruption horloge le processus revient à l'état prêt. S'il fait appel système pour attendre le transfert des données depuis le disque par exemple ou tout autre type d'événement il passe à l'état **bloqué**. Le processus revient à l'état prêt suite à l'arrivée de l'événement attendu (comme interruption DMA par exemple).

3. Les Threads : Le modèle processus décrit précédemment est un programme qui s'exécute selon un chemin unique (compteur ordinal). On dit qu'il a un flot de contrôle unique (un seul thread). De nombreux systèmes d'exploitation modernes offrent la possibilité d'associer à un même processus plusieurs chemins d'exécution (multithreading, multiflot d'exécution).

Un thread est une unité d'exécution, rattachée à un processus, chargée d'exécuter une partie du processus. Un processus est vu comme étant un ensemble de ressources (espace d'adressage, fichiers, périphériques...) que ses threads (flots de contrôle ou processus légers) partagent. Lorsqu'un processus est créé, un seul flot d'exécution (thread) est associé au processus. Ce thread peut en créer d'autres.

- Chaque thread a :

- un identificateur unique
- une pile d'exécution
- des registres (un compteur ordinal)
- un état...

Le multithreading permet l'exécution simultanée ou en pseudo-parallèle de plusieurs parties d'un même processus. Ce qui donne plusieurs avantages :

- Réactivité (le processus peut continuer à s'exécuter même si certaines de ses parties sont bloquées),
- Partage de ressources (facilite la coopération, améliore la performance),
- Économie d'espace mémoire et de temps. Il faut moins de temps pour : créer, terminer un thread (sous Solaris, la création d'un processus est 30 fois plus lente que celle d'un processus),

4. Notion de ressource : Nous appelons ressource toute entité dont un processus a besoin pour s'exécuter. Tout les composant matériels et logiciels peuvent être appelé ressource : (CPU, Memoire, Bus). Nous appelons ressource aussi une variable partagée par des threads, une adresse mémoire, un fichier sur disque. Une ressource peut être caractérisé par :

- Utilisable par plusieurs processus en même temps (donnée en lecture seule).
- Utilisable par un seul processus à la fois (variable partagée par plusieurs processus ou threads, un fichier en écriture, imprimante).

4.1 Partage de ressource

L'utilisation d'une ressource passe par trois étapes : (1) Demande d'allocation, (2) Utilisation, (3) restitution. Si la ressource est utilisable par un seul processus, on doit s'assurer qu'elle est libre avant qu'elle soit allouée à un processus. Cette ressource est appelée **ressource critique**.